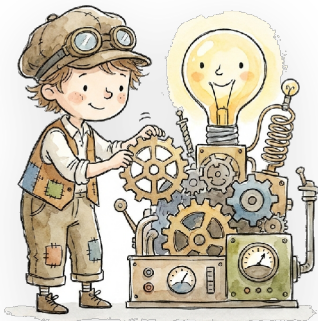




CALCULUS FOR OPTIMIZATION

The Chain Rule and Backpropagation



YOUR GUIDE

Let's get started.

A neural network is a function wrapped inside a function wrapped inside a function, which might be ninety layers deep. Training it means answering one question millions of times: **if we nudge this one weight buried in layer three, how much does the final loss change?** That question sounds impossibly hard. The weight is separated from the loss by dozens of matrix multiplies and nonlinearities, however, a single, standard rule from first-year calculus answers it exactly and cheaply. That rule is the **chain rule**, and the algorithm that applies it layer by layer is called **backpropagation**. This chapter explores how one composes into

the other, and why that composition forms the mathematical heart of everything that learns.

In the [previous chapter](#), we built the derivative, which is the instantaneous slope, and the gradient, the vector of partial derivatives that points uphill on the loss surface. That gave us a compass. What we did *not* explain is how we compute that compass for a function made of many nested pieces. Addressing that computation is our main focus in this section.

The puzzle: a weight far from the loss

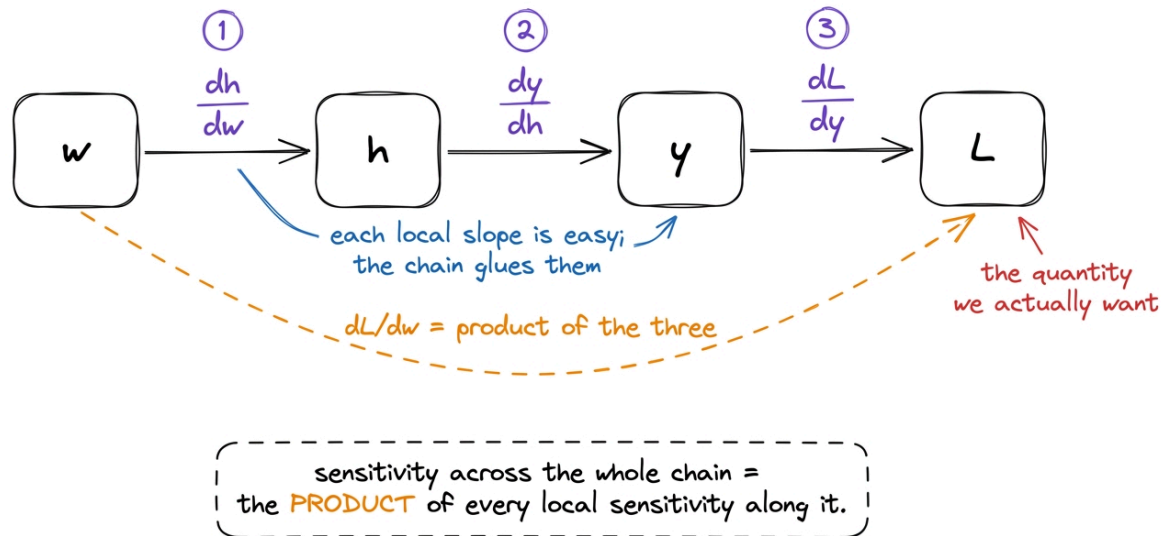
We can make the difficulty concrete before we resolve it. Suppose the loss L depends on a prediction y , which depends on a hidden value h , which depends on a weight w . Written out, we have $L = L(y(h(w)))$. We want to find dL/dw , which is the sensitivity of the loss to that weight, however, we have no formula that connects them directly. We only have the *links in the chain*: how L responds to y , how y responds to h , and how h responds to w . We can differentiate each link on its own. The question is how we can glue those local slopes into one global slope.

The chain rule serves as the glue, and its formulation is quite straightforward:

$$dL/dw = dL/dy \cdot dy/dh \cdot dh/dw$$

We multiply the local derivatives along the chain. The Leibniz notation makes it look like the dy 's and dh 's "cancel" like fractions. They are not fractions but limits, however, that cancellation is a genuine, provable consequence of the limit definition. This is a rare case where the informal intuition is also mathematically correct. This concept forms the foundation of the process. Every complication in backpropagation that follows is essentially bookkeeping on top of this one line.

The chain rule = multiply the local slopes



The chain rule turns one hard global derivative into a product of easy local derivatives, one per link.

Napkin arithmetic on a two-link chain

Abstract rules can be difficult to internalize, so we can put real numbers on a tiny composed function to illustrate the concept. Take $h = 3w$, then $y = h^2$, and a loss $L = y$. For example, pick $w = 2$. During the forward pass, that gives $h = 6$, $y = 36$, and $L = 36$.

Now the three local slopes. $dh/dw = 3$ (the slope of $3w$). $dy/dh = 2h = 12$ (the slope of h^2 at $h = 6$). $dL/dy = 1$. Multiply them along the chain:

$$dL/dw = dL/dy \cdot dy/dh \cdot dh/dw = 1 \cdot 12 \cdot 3 = 36$$

Thus, increasing w by a small amount changes L at a rate of **36 to 1**. We can sanity-check that by brute force, nudging w from **2** to **2.001**. As a result, the new $h = 6.003$, $y = 36.036009$, so L rose by about **0.036**, and $0.036 / 0.001 = 36$. The chain rule and the brute-force finite difference agree to the digit. That agreement represents the core mechanism: local slopes, multiplied, give the true global slope.¹

When a value fans out: add the gradients

A single chain is the easy case. Real networks are not chains, however, they are *graphs*, meaning one value almost always feeds into several places at once. Every neuron sends its output to many neurons in the next layer. If we nudge that neuron, the loss feels the change through *every* one of those paths. The rule for this is a common source of confusion: **when a value fans out to several downstream paths, its gradient is the sum of the gradients coming back along each path**. It is not the max or the average,

but the sum. This is just the multivariable chain rule, and it follows from the fact that a small change in the value perturbs every path at once, and the perturbations add.

We can put small numbers on it to observe this behavior. Suppose a value $u = 2$ is used twice: once as $p = 3u$ and once as $q = u^2$, and the loss is $L = p + q$. Then we have $dL/dp = 1$ and $dL/dq = 1$. Along the first path the local slope is $dp/du = 3$, and along the second it is $dq/du = 2u = 4$. The gradient at u is the *sum*, $1 \cdot 3 + 1 \cdot 4 = 7$, rather than either branch alone. For a brute-force check, we nudge u from 2 to 2.001, and L moves from 10 to 10.007002, which is a rate of about 7 to 1. If we had only followed one branch, we would have reported 3 or 4 and computed an incorrect result. In backpropagation, the same thing happens at every neuron whose output feeds multiple downstream neurons. The deltas flowing back from all of them *accumulate* at that node before it passes anything further back.

One neuron, forward and backward

Constants / Parameters:

x = input vector

w = weights vector

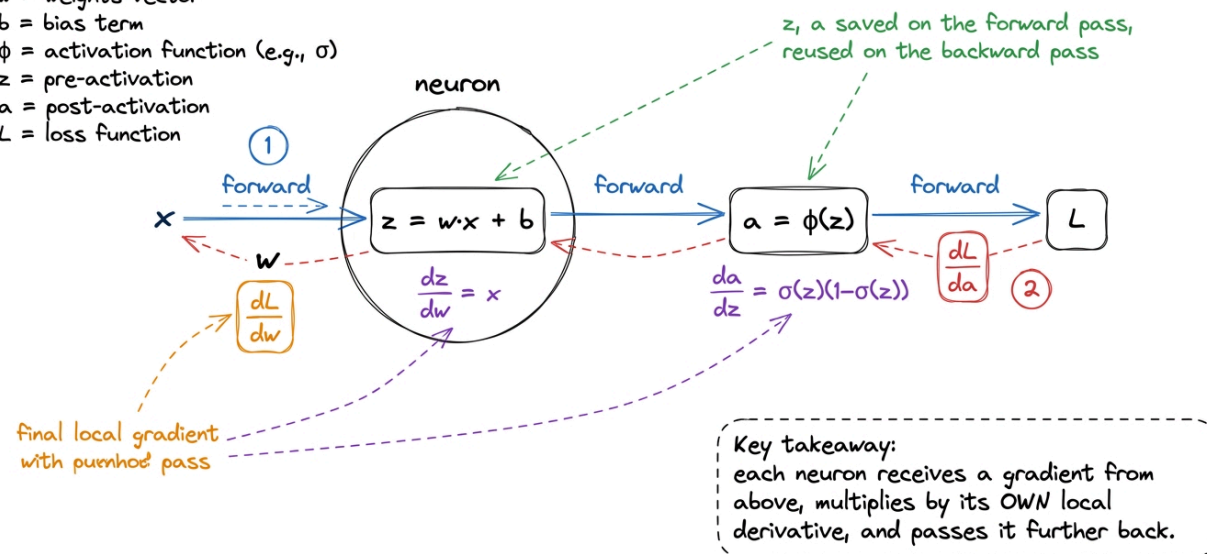
b = bias term

ϕ = activation function (e.g., σ)

z = pre-activation

a = post-activation

L = loss function



When one value feeds several paths, its gradient is the sum of the gradients returning along every path, not any single branch.

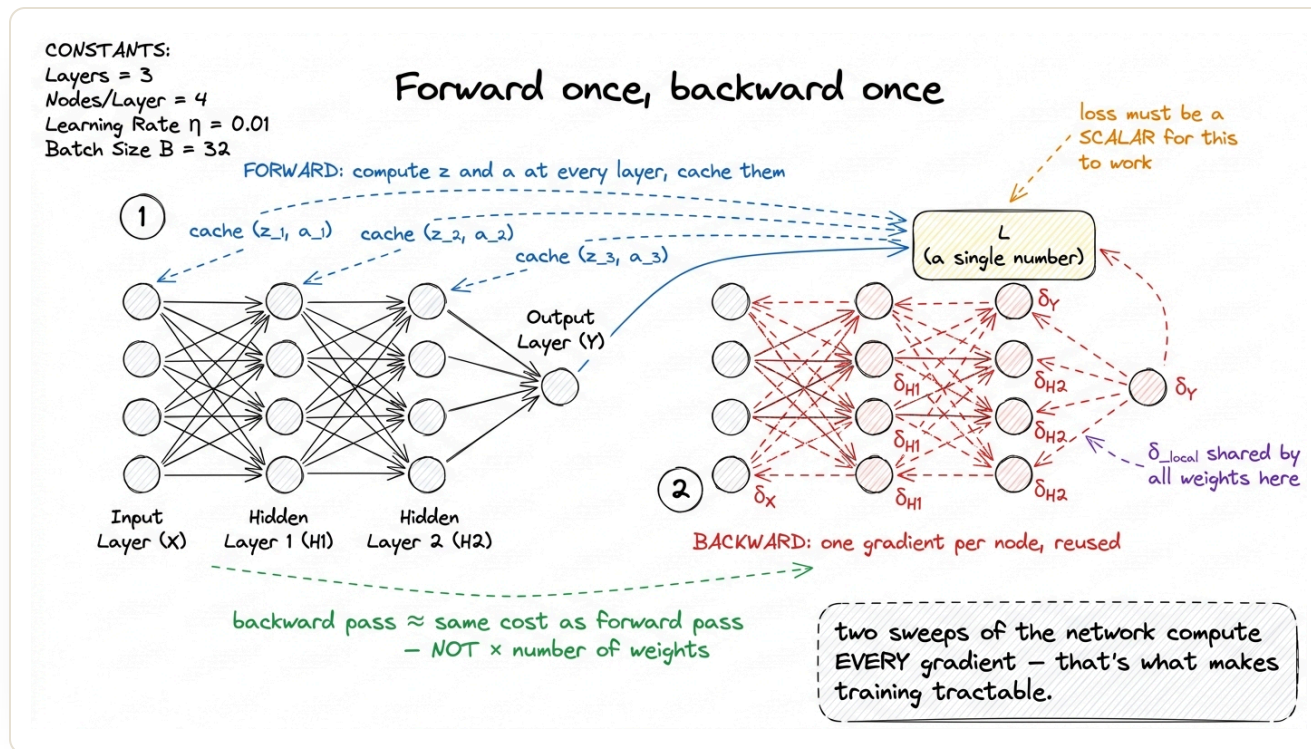
From a chain to a neuron

A real neuron operates similarly. It computes a weighted sum $z = w \cdot x + b$, then squashes it through a nonlinear **activation** $a = \phi(z)$, however, we will use the **sigmoid** $\sigma(z) = 1/(1 + e^{-z})$ as an example. We feed a into a loss, and to update w we want to compute dL/dw . We apply the chain rule, which gives us three links again:

$$dL/dw = dL/da \cdot da/dz \cdot dz/dw$$

We read it right to left, which is the way backpropagation actually runs. First, we have $dz/dw = x$, where the pre-activation's slope with respect to the weight is just the input.² Next is $da/dz = \sigma(z) \cdot (1 - \sigma(z))$, as the sigmoid has a standard, straightforward derivative. Finally, dL/da comes from whatever loss sits on top. We multiply the three terms, and we obtain the exact gradient for that weight without requiring a single formula for the entire composed function.

We can observe the structure of this computation. To obtain the gradient at a specific weight, we need one quantity passed *down from above* (dL/da , representing how much the loss depends on this neuron's output) and one quantity that is *local* ($da/dz \cdot dz/dw$, representing how this neuron transforms its inputs). Backpropagation relies on this interaction, repeating it at every neuron, moving from the output end backward to the input end.



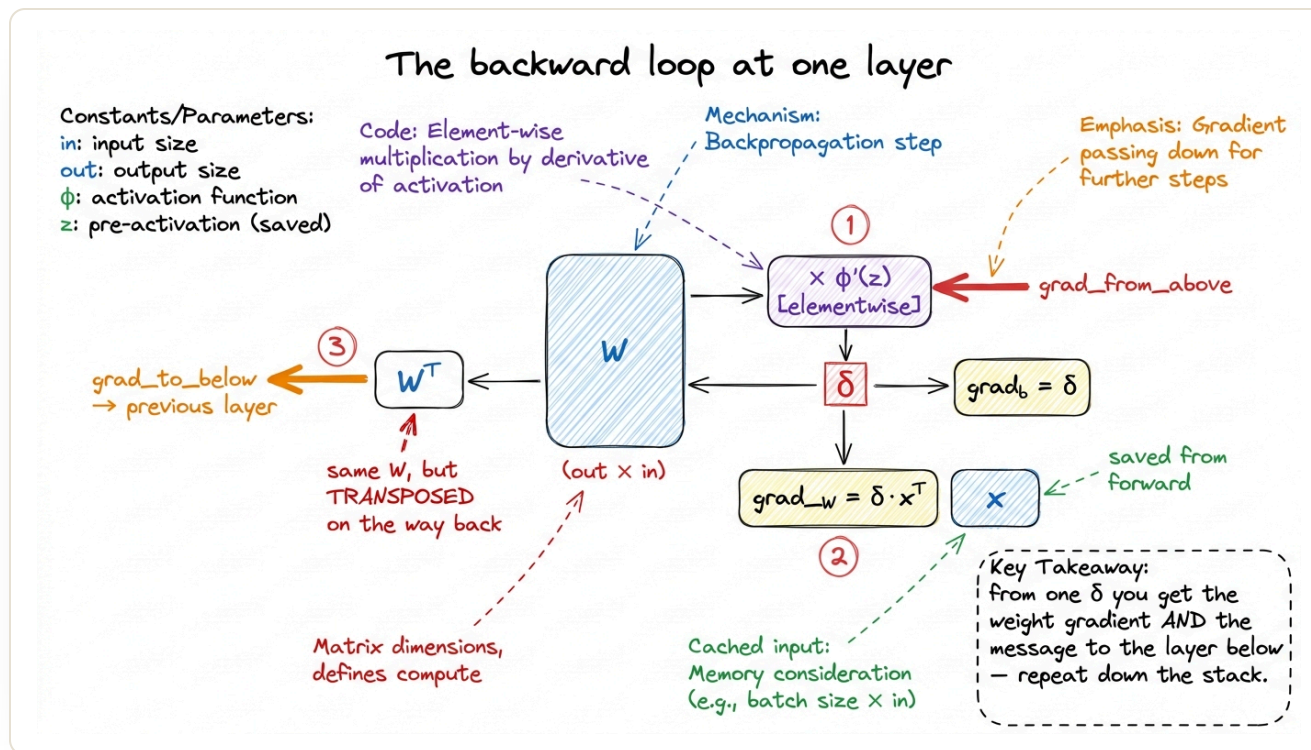
A neuron's weight gradient is the upstream gradient times two local slopes; the forward pass caches what the backward pass consumes.

Why one backward pass, not millions of forward ones

We can now examine how this mathematical identity functions as the engine of modern AI. For example, we can imagine a network with a million weights. The naive way to obtain every gradient is to perturb each

weight, run a full forward pass, and measure how L moved. This would require a million forward passes per training step; however, such an approach is computationally infeasible.

The chain rule provides us with a significantly better approach. Because the *upstream* gradient dL/da is shared by everything feeding into that neuron, we can compute it **once** and reuse it for all the weights at that neuron. Furthermore, we can pass it further back so the previous layer can reuse it as well. We sweep forward once to compute all the values, and then we sweep backward *once* to compute all the gradients. Note that the cost of the backward pass is roughly the same as the forward pass.³ We require two passes in total, rather than a million. This single-pass reuse explains how a hundred-billion-parameter model can be trained in practice.



Backprop is two network sweeps total: forward to cache values, backward to distribute one reusable gradient per node. Cost is independent of parameter count.

The delta that flows backward

We can now define the quantity that flows through the network. We define δ (delta) at a neuron as dL/dz , which represents the sensitivity of the loss to that neuron's *pre-activation*. This value is the primary signal that backpropagation moves around. Once we compute a layer's δ , two results

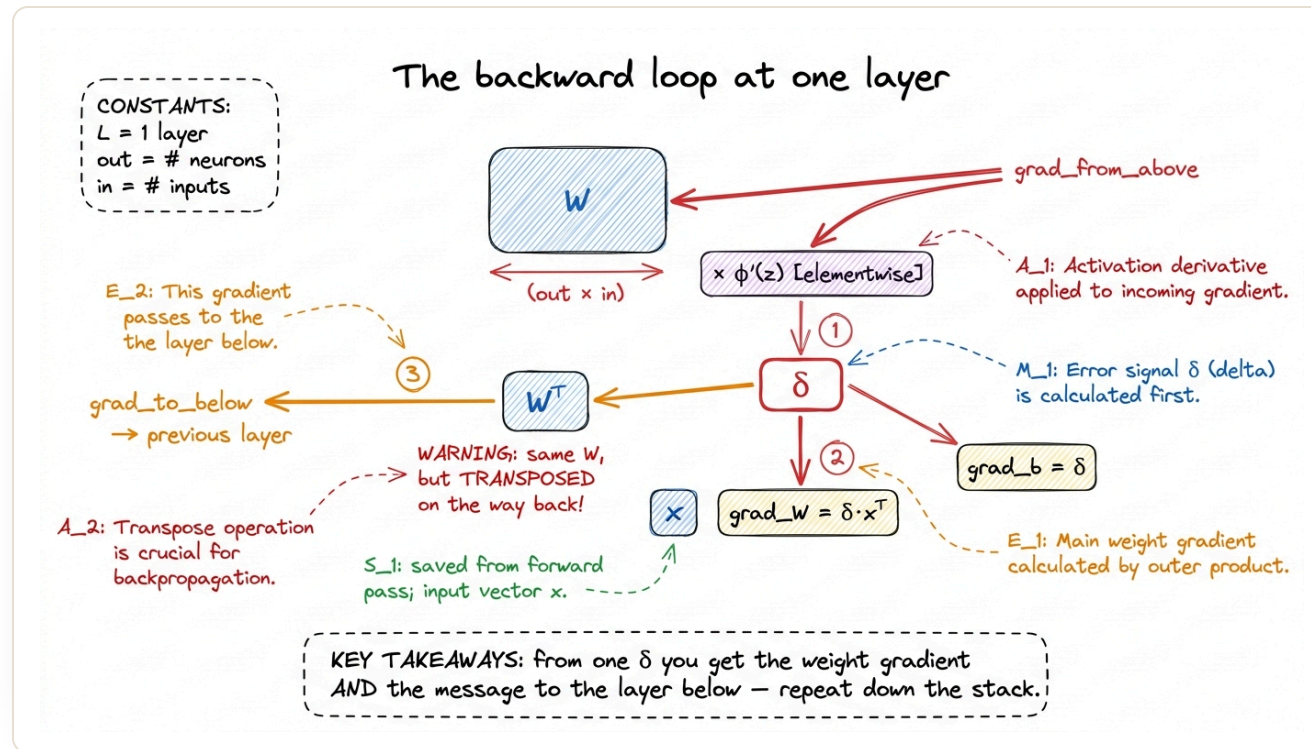
follow immediately. First, the gradient for that layer's weights is δ times the layer's cached inputs. Second, the δ we pass to the *previous* layer is this δ pushed back through the weights and through the previous layer's activation derivative.

In matrix form, for a layer with weights W , we can write the backward step compactly:

```
# forward pass cached: x (input), z = W @ x + b, a = phi(z)
delta = grad_from_above * phi_prime(z)    # elementwise: dL/dz for this layer
grad_W = delta @ x.T                      # gradient for the weights
grad_b = delta                            # gradient for the bias
grad_to_below = W.T @ delta               # the delta handed to the previous la
```

This process requires four lines. The value `grad_from_above` arrives from the layer in front, and we multiply it elementwise by the activation's slope `phi_prime(z)` to obtain this layer's `delta`. From `delta`, we can read off the weight gradient (`delta @ x.T`), the bias gradient, and the message to send backward (`W.T @ delta`). The forward pass used W ; however, the backward pass uses W **transposed**. We use the same weights, run in reverse, routing the error back to where it originated.⁴ If we chain this loop from the

output layer down to the first, we have successfully computed every gradient in the network.



At each layer, the elementwise product with $\phi'(z)$ makes δ , from which the weight gradient and the backward message both follow.

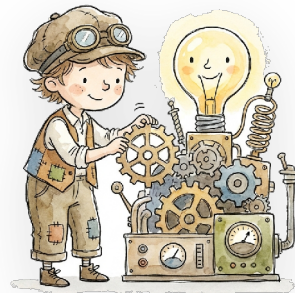
The bold takeaway

At its core, we can summarize backpropagation in one sentence: **the gradient of the loss with respect to any weight is the product of the local**

derivatives along the path from that weight to the loss, and the whole network's worth of these products is computed in a single backward sweep that costs about the same as one forward pass. This is not a new principle or a special algorithm invented for neural networks; however, it is simply the standard chain rule, $d/dx \ f(g(x)) = f'(g(x)) \cdot g'(x)$, organized so that shared sub-results are computed once and reused. The critical factor is the computational cost: we require **two passes, not one-per-parameter**. This efficiency is the difference between training a billion-parameter model in hours and never being able to train it at all.

There is one important caveat worth carrying forward. Multiplying many local slopes along a deep chain is also *fragile*. If each slope is slightly less than one, the product shrinks toward zero after many layers, which is known as the **vanishing gradient** problem; however, if each slope is slightly more than one, the product explodes. For example, the sigmoid's derivative never exceeds 0.25, so a stack of sigmoids can reduce the gradient to zero before it reaches the early layers.⁵ The chain rule is mathematically exact, but the numbers it multiplies still have to remain stable.

We now have the exact gradient at every weight and the efficient algorithm to compute it. However, we have *not* yet taken an optimization step. We still need to turn that gradient into an actual weight update, choose a learning rate, and loop the entire process until the loss stops falling. That optimization story, where these gradients get *used*, is covered in [gradient descent and backpropagation](#). Before we leave calculus behind, we will also explore the mirror image of the derivative. Accumulating tiny pieces into a whole, which underlies probability and expected values, is waiting in [integral calculus foundations for ML](#).



CHAPTER COMPLETE

Nicely done. You reached the end of this one.

[NEXT CHAPTER](#) **Integral Calculus Foundations for ML****0**

Did this chapter land for you?

Mark chapter as done

Rate this chapter ★ ★ ★ ★ ★

What did you think of this chapter?

A takeaway, a question, a moment that clicked...



Be kind and specific. 0/2000

Post*No notes yet — be the first to share what you took away.*[< Differential Calculus: ...](#)[◇ Back to book](#)[Integral Calculus Found... >](#)